

Inventory Movement System Implementation

Overview

Complete implementation of an inventory movement tracking system for the CRTLPyme SaaS platform. This system allows tenants to track all stock movements (entries, exits, and adjustments) with proper audit trails and tenant isolation.

Implementation Date

November 8, 2025

Components Implemented

1. Database Schema (prisma/schema.prisma)

New Model: InventoryMovement

```
model InventoryMovement {
  id          String      @id @default(cuid())
  tenantInventoryId String
  type        MovementType
  quantity     Int
  reason       String?
  notes        String?
  createdBy    String
  tenantId     String
  createdAt    DateTime   @default(now())

  // Relations
  tenantInventory TenantInventory @relation(...)
  user            User           @relation(...)
  tenant          Tenant         @relation(...)
}
```

New Enum: MovementType

- ENTRY : Stock entries (purchases, restocking)
- EXIT : Stock exits (adjustments, losses)
- ADJUSTMENT : Inventory corrections

2. API Endpoints (app/api/inventory/movements/route.ts)

GET /api/inventory/movements

Purpose: List inventory movements with filtering

Query Parameters:

- tenantInventoryId (optional): Filter by specific product
- type (optional): Filter by movement type (ENTRY/EXIT/ADJUSTMENT)
- startDate (optional): Filter by start date
- endDate (optional): Filter by end date
- limit (optional): Limit results (default: 50)

Response:

```
{
  "movements": [...],
  "stats": {
    "totalMovements": 100,
    "entriesCount": 60,
    "exitsCount": 30,
    "adjustmentsCount": 10,
    "totalEntryQuantity": 500,
    "totalExitQuantity": 200
  }
}
```

Authentication: Required (NextAuth session)**Authorization:** All authenticated users can view their tenant's movements**POST /api/inventory/movements****Purpose:** Register new movement and update stock**Request Body:**

```
{
  "tenantInventoryId": "string",
  "type": "ENTRY | EXIT | ADJUSTMENT",
  "quantity": number,
  "reason": "string (required, max 255)",
  "notes": "string (optional, max 500)"
}
```

Response:

```
{
  "id": "...",
  "type": "ENTRY",
  "quantity": 50,
  "previousStock": 100,
  "newStock": 150,
  "reason": "Purchase from supplier",
  "notes": "Invoice #12345",
  "tenantInventory": {...},
  "user": {...},
  "createdAt": "2025-11-08T..."
}
```

Authentication: Required (NextAuth session)**Authorization:** ADMIN, INVENTARIO, PROVEEDOR roles only**Business Logic:**

1. Validates product exists and belongs to tenant
2. Calculates quantity change based on movement type:
 - ENTRY: Adds to stock (+quantity)
 - EXIT: Removes from stock (-quantity)
 - ADJUSTMENT: Can add or remove ($\pm$ quantity)
3. Prevents negative stock

4. Updates stock in transaction

5. Creates audit log entry

3. UI Page (`app/admin/inventory/movements/page.tsx`)

Features

- **Dashboard View:** Shows movement statistics (entries, exits, adjustments)
- **Filter Panel:**
 - Type filter (All/Entries/Exits/Adjustments)
 - Date range filter (start and end dates)
 - Clear filters button
- **Movement List:** Table with all movements showing:
 - Date/time
 - Type (with colored badges)
 - Product details
 - SKU
 - Quantity (color-coded: green for +, red for -)
 - Reason and notes
 - User who registered
- **Register Movement Dialog:**
 - Product selector (with current stock)
 - Movement type selector (with icons)
 - Quantity input
 - Reason input (required)
 - Notes textarea (optional)
 - Form validation with Zod

User Experience

- Real-time validation
- Toast notifications on success/error
- Loading states
- Empty states with helpful messages
- Responsive design (mobile-friendly)

Security & Best Practices

Tenant Isolation

- All queries filtered by `tenantId` from session
- Products must belong to the tenant
- Cross-tenant access prevented

Authentication & Authorization

- NextAuth session required for all operations
- Role-based access control (RBAC)
- Only ADMIN, INVENTARIO, and PROVEEDOR can create movements

Data Validation

- Zod schemas for type-safe validation
- Prevents negative stock
- Input sanitization
- Max length constraints on text fields

Audit Trail

- All movements logged with user and timestamp
- Audit log entries created for each movement
- Previous and new stock values recorded

Transaction Safety

- Stock updates use Prisma transactions
- Atomic operations prevent race conditions
- Rollback on errors

Database Migration

The schema changes will be applied automatically by the CI/CD pipeline on deployment:

```
npx prisma migrate dev --name add_inventory_movement_model
```

Testing Checklist

API Testing

- ☐ GET movements without filters
- ☐ GET movements with type filter
- ☐ GET movements with date range
- ☐ POST ENTRY movement (increases stock)
- ☐ POST EXIT movement (decreases stock)
- ☐ POST ADJUSTMENT movement
- ☐ Prevent negative stock on EXIT/ADJUSTMENT
- ☐ Verify tenant isolation
- ☐ Test authorization (role check)

UI Testing

- ☐ View movements page loads correctly
- ☐ Statistics cards display correct data
- ☐ Filters work as expected
- ☐ Register movement dialog opens
- ☐ Form validation works
- ☐ Movement created successfully
- ☐ Toast notifications appear
- ☐ Table updates after creating movement
- ☐ Date formatting correct (Spanish locale)

- [] Responsive on mobile devices

Deployment

Prerequisites

- Database must be accessible
- Environment variables configured (DATABASE_URL)
- CI/CD pipeline configured

Deployment Steps

1. Commit changes to main branch
2. CI/CD pipeline automatically:
 - Runs Prisma migrations
 - Builds Next.js application
 - Deploys to production

Usage Examples

Registering a Purchase (Entry)

1. Navigate to `/admin/inventory/movements`
2. Click “Registrar Movimiento”
3. Select product
4. Choose “Entrada” type
5. Enter quantity (e.g., 100)
6. Enter reason: “Compra a proveedor ABC”
7. Add notes: “Factura #12345” (optional)
8. Click “Registrar Movimiento”
9. Stock automatically increased

Registering a Loss (Exit)

1. Click “Registrar Movimiento”
2. Select product
3. Choose “Salida” type
4. Enter quantity (e.g., 5)
5. Enter reason: “Producto dañado”
6. Click “Registrar Movimiento”
7. Stock automatically decreased

Filtering Movements

- Select movement type from dropdown
- Set date range for specific period
- View statistics update automatically

Future Enhancements

Potential Improvements

- [] Export movements to Excel/PDF
- [] Batch movement registration
- [] Movement templates for common operations
- [] Advanced analytics dashboard
- [] Integration with purchase orders
- [] Barcode scanning for quick registration
- [] Mobile app for warehouse operations
- [] Real-time notifications for low stock
- [] Movement approval workflow
- [] Integration with accounting system

Related Files

Modified

- `prisma/schema.prisma` - Added InventoryMovement model and MovementType enum

Created

- `app/api/inventory/movements/route.ts` - API endpoints
- `app/admin/inventory/movements/page.tsx` - UI page

Notes

- The system maintains backward compatibility with existing StockAdjustment model
- All movements are immutable (no edit/delete) for audit integrity
- Stock updates are atomic using Prisma transactions
- Date/time formatting uses Spanish locale (es-CL)
- UI uses shadcn/ui components for consistency

Support

For issues or questions, contact the development team or create an issue in the repository.